No.	Questions
	What is List?
	List Length?
	How to Check if Item Exists?
	List Comprehension?
	Loop Lists?
	Difference between List and Tuples in Python?
	What is the difference between an array and a list?
	list() Constructor?
	How to Convert a list into numpy?

Ques. What is List?

- Lists are used to store multiple items in a single variable.
- List items are **ordered**, **changeable**, and **allow duplicate** values.
- In Python lists are written with **square brackets[]**, separated by commas.
- The list is **changeable**, meaning that we can change, add, and remove items in a list after it has been created.
- list Allow Duplicates values. **Ex:-** list = ["apple", "banana", "cherry", "apple", "cherry"]
- List items can be of any data type. **Ex:-** list = ["abc", 34, True, 40, "male"]

```
my_list = ["apple", "banana", "cherry", "banana"]
my_list = [1, "Hello", 3.4]
my_list = ["mouse", [8, 4, 6], ['a']]  # A list can also have another list as
                                       # an item. This is called a nested list.
print(my_list)
```

Ques. List Length

- use the **len()** function:

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist)) # Output:- 3
```

Ques. How to Check if Item Exists?

```
thislist = ["apple", "banana", "cherry"]
if "apple" in thislist:
  print("Yes, 'apple' is in the fruits list")

# Output:- Yes, 'apple' is in the fruits list
```

Ques. List Comprehension?

- List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.
- **Syntax** newlist = [expression for item in iterable if condition == True]
  - **expression:** What you want to put in the new list (often uses the item)
  - **for item in iterable:** Loop through each item in an iterable (like a list, range, etc.)
  - **if condition (optional):** Only include items that meet this condition

```
# Without list comprehension you will have to write a for statement with a
conditional test inside.
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = []

for x in fruits:
  if "a" in x:
    newlist.append(x)

print(newlist)

# With list comprehension you can do all that with only one line of code:
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits if "a" in x]

print(newlist)  # Output:- ['apple', 'banana', 'mango']
```

```
# Example 1: Create a list of squares from 0 to 4
squares = [x**2 for x in range(5)]
print(squares)  # Output: [0, 1, 4, 9, 16]

# Equivalent without list comprehension:
squares = []
for x in range(5):
    squares.append(x**2)
```

```
# Example 2: Filter even numbers from a list
evens = [x for x in range(10) if x % 2 == 0]
print(evens)  # Output: [0, 2, 4, 6, 8]
```

Ques. Why use list comprehension?

- More compact and readable code
- Often faster than a manual loop

Ques. Loop Lists?

```
# Loop Through a List
thislist = ["apple", "banana", "cherry"]
for x in thislist:
  print(x)

Output:-
apple
banana
cherry
--------------------------------------------------------------------------------

# Loop Through the Index Numbers
# You can also loop through the list items by referring to their index number.
#Use the range() and len() functions to create a suitable iterable.
thislist = ["apple", "banana", "cherry"]
for i in range(len(thislist)):
  print(thislist[i])

Output:-
 apple
 banana
 cherry
```

- Using a **While Loop**

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
  print(thislist[i])
  i = i + 1

Output:-
apple
banana
cherry
```

- Looping Using **List Comprehension**

```
thislist = ["apple", "banana", "cherry"]
[print(x) for x in thislist]

Output:-
apple
banana
cherry
```

Ques. What is the difference between an array and a list?

#	List	Array
1	It Contains elements of different data types	It Contains elements of same data types
2	Cannot handle arithmetic operations	Can handle arithmetic operations
3	We can print the entire list without the help of an explicit loop	To print or access array elements, we will require an explicit loop
4	It consumes a large memory	It is a more compact in memory size comparatively list.

Ques. Difference between List and Tuples in Python?

List	Tuples
List is mutable. i.e they can be edited.	Tuple is immutable. (tuples are lists which can’t be edited).
List iteration is slower and is time consuming.	Tuple iteration is faster.
List is useful for insertion and deletion operations.	Tuple is useful for readonly operations like accessing elements.
List has a large memory.	Tuple has a small memory.
List is stored in two blocks of memory (One is fixed sized and the other is variable sized for storing data)	Tuple is stored in a single block of memory.
List provides many in-built methods.	Tuples have less in-built methods.
List operations are more error prone	Tuples operations are safe.
A list has data stored in square brackets [] brackets. For example, list_1 = [10, ‘Chelsea’, 20]	A tuple has data stored in parantheses () brackets. For example, tup_1 = (10, ‘Chelsea’ , 20)

Ques. list() Constructor?

- It is also possible to use the **list()** constructor to make a new list.

```
thislist = list(("apple", "banana", "cherry"))
print(thislist)

Output:- ['apple', 'banana', 'cherry']
```

How to Convert a list into numpy?

```
# Firstly install numpy
pip install numpy

# Example list
import numpy as np
my_list = [1, 2, 3, 4, 5]
# Convert list to NumPy array
my_array = np.array(my_list)
print(my_array)  # Output: [1 2 3 4 5]
```